Proximal Policy Optimization for Atari Pitfall-V5 game (PPO) Algorithmic Comparison with Deep Q-Networks and Intrinsic Curiosity

Angelo Ibañez, David Ariza, Cristhian Truque
Systems Engineering
Universidad Distrital Francisco José de Caldas
Email: aaibanezh@udistrital.edu.co, dfarizaa@udistrital.edu.co, cstruquecl@udistrital.edu.co

***Abstract*—Deep Reinforcement Learning (DRL) has achieved superhuman performance in many Atari games, yet environments with sparse and delayed rewards remain a major challenge. This paper investigates and compares the performance of two DRL approaches, Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO), in Pitfall! using the ALE/Pitfall-v5 environment. We systematically evaluate different hyperparameter configurations and analyze the ability of both algorithms to overcome the game's hard-exploration problem. Our results show that standard DQN optimization often converges to a passive local minimum that avoids negative rewards, while PPO exhibits more stable policy updates but struggles to discover meaningful rewards due to the sparse-reward structure of the environment. Additionally, introducing a count-based intrinsic curiosity mechanism encourages the DQN agent to explore novel sub-surface states, highlighting the importance of intrinsic motivation in sparse-reward scenarios. Despite these improvements, both approaches remain limited by the high computational cost and long training times required to achieve consistent positive performance in this environment.**

***Index Terms*—Deep Q-Networks, Deep Learning, Arcade Learning Enviroment, Pitfall, Sparse Rewards, Intrinsic Curiosity, Proximal Policy Optimization (PPO).**

I. Introduction

Deep Reinforcement Learning (DRL) algorithms such as Deep Q-Networks (DQN) [1] and Proximal Policy Optimization (PPO) [2] have demonstrated superhuman performance across many Atari 2600 environments. However, environments with sparse or delayed rewards remain a significant open challenge. When positive feedback is rare, agents struggle to discover rewarding trajectories, often converging to degenerate behaviors that minimize penalties instead of maximizing long-term returns. In value-based approaches such as DQN, limited exploration can trap the agent in passive local optima, while policy-gradient methods such as PPO may suffer from overly conservative updates that hinder effective exploration.

Pitfall! (ALE/Pitfall-v5) is a representative example of this problem. The player must navigate a jungle environment while avoiding crocodiles, scorpions, and tar pits, and collecting treasures distributed across the map. Rewards are sparse, delayed, and partially hidden underground, making successful exploration extremely difficult. A randomly acting agent has a very low probability of discovering positive rewards within a standard training budget, turning the environment into a benchmark for hard exploration in reinforcement learning.

This paper presents our experimental study comparing DQN and PPO in ALE/Pitfall-v5. We evaluate a standard DQN baseline, perform structured OFAT hyperparameter sweeps, explore additional parameter combinations, and implement a count-based intrinsic curiosity mechanism to improve exploration. In parallel, we analyze PPO using Atari-specific preprocessing and convolutional policy networks to examine how stable policy optimization behaves under sparse-reward conditions. All experiments are reproducible through the accompanying repository using fixed random seeds and controlled environment management with Poetry and pyenv.

II. Background

A. Deep Q-Networks

DQN approximates the action-value function $Q(s, a)$ with a convolutional neural network (CNN), trained via temporal-difference learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big] \quad (1)$$

Training stability is achieved through two key mechanisms: an *experience replay buffer* that decorrelates successive samples, and a periodically updated *target network* that provides stable bootstrapping targets. The agent follows an ε-greedy policy during training, decaying ε from 1.0 to a small final value over a fraction of the total training steps.

B. Sparse Reward Environments

Sparse reward environments present a fundamental exploration challenge: the agent must discover rewarding states before it can learn to reach them. Classical epsilon-greedy exploration scales poorly when rewards require long action sequences to reach. This problem is particularly acute in Pitfall, where the agent must execute dozens of precise actions—avoiding enemies, crossing logs, swinging on vines—before encountering any reward signal.

C. Count-Based Curiosity

Count-based exploration maintains visit counts $N(s)$ for each state and augments the extrinsic reward with an intrinsic bonus:

$$r_{\text{intrinsic}}(s) = \frac{\beta}{\sqrt{N(s)}} \quad (2)$$

This encourages visits to less-explored states. In high-dimensional environments such as Atari, exact state counts are infeasible. A common approximation uses a hash of a downsampled observation as a proxy state identifier, enabling tractable count maintenance.

D. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy-gradient reinforcement learning algorithm introduced by John Schulman and collaborators as a simpler and more stable alternative to Trust Region Policy Optimization (TRPO). PPO directly optimizes the agent's policy by updating the probability distribution over actions while constraining the magnitude of policy changes between updates. This is achieved through a clipped surrogate objective that prevents excessively large updates, improving training stability and reducing the risk of catastrophic performance collapse. PPO combines policy optimization with value-function estimation and typically uses Generalized Advantage Estimation (GAE) to reduce variance during training. Due to its balance between simplicity, stability, and performance, PPO has become one of the most widely used reinforcement learning algorithms for high-dimensional environments, including continuous control tasks and Atari benchmarks. However, despite its strong stability properties, PPO can struggle in environments with sparse rewards and difficult exploration requirements, since conservative policy updates may limit the agent's ability to discover rewarding trajectories.

III. Experimental Setup

A. Environment and Preprocessing

We use `ALE/Pitfall-v5` through Gymnasium with the standard Atari preprocessing pipeline for both DL algorithms:

- Grayscale conversion and frame resize to 84×84 pixels
- Frame skip of 4 (each action repeated for 4 consecutive frames)
- Stack of the last 4 frames as the observation ($4 \times 84 \times 84$)
- Terminal-on-life-loss during training
- Reward clipping to $[-1, +1]$ during training; disabled at evaluation

B. Implementation

All DQN agents use Stable-Baselines3 v2.7.1 with the `DQN` implementation and `CnnPolicy`. PPO experiments were implemented from scratch in PyTorch using an actor-critic architecture with convolutional neural networks adapted for Atari environments. The PPO implementation includes clipped policy optimization, Generalized Advantage Estimation (GAE), entropy regularization, and Atari preprocessing with grayscale observations, frame stacking, and frame skipping following the challenge specifications. All experiments were executed on CPU without GPU acceleration, limiting the feasible training budget and total number of environment interactions. The complete codebase is available in the project repository with reproducible instructions using pyenv 3.11.10 and Poetry.

C. Hyperparameter Search Strategy

We adopt a structured **One-Factor-At-A-Time (OFAT)** strategy for Phase 1. One hyperparameter is varied per experiment while all others are held at baseline values. This allows isolated measurement of each factor's contribution. Phase 2 combines the best-performing factors from Phase 1, this mehtod was also applied in the PPO method. Table II summarizes the search space.

TABLE I
DQN Hyperparameter Search Space

Hyperparameter	Values Tested
Learning rate α	5×10^{-5}, 1×10^{-4}, 2×10^{-4}
Replay buffer size	50 000; 100 000; 200 000
Learning starts	10 000; 25 000; 50 000
Batch size	32; 64; 128
Discount factor γ	0.99; 0.995
Train frequency	4 steps/update
Target update interval	1 000; 2 000; 5 000
Exploration fraction	0.15; 0.30; 0.50
Final ε	0.001; 0.01; 0.02; 0.05; 0.10

TABLE II
PPO Hyperparameter Search Space

Hyperparameter	Values Tested
Learning rate α	1×10^{-4}, 2.5×10^{-4}, 5×10^{-4}
Horizon	52; 1 024; 2 048
Number of epochs	4; 6; 10
Mini-Batch size	32; 64; 128
Discount factor γ	0.99; 0.995
Gae λ	0.9; 0.95; 0.97
Clip epsilon ϵ	0.1; 0.2
Entropy coefficient c_2	0.001; 0.01; 0.02
Value-loss coefficient c_1	0.5; 1.0
Gradient norm clip	0.5; 1.0; none

D. Evaluation Protocol

DQN Algorithm: Phase 1 experiments use 300 000 timesteps with three independent random seeds (42, 100, 2026). Phase 2 uses 500 000 timesteps. Phase 3 (curiosity) uses 500 000 timesteps. The score reported for each run is the mean episode reward over the last episodes stored in SB3's internal buffer at the end of training. Three seeds are used to report variance and avoid reporting statistical outliers.

PPO Algorithm: The initial experiments were conducted using 5,000,000 training timesteps with three independent random seeds (42, 100, and 2026) to evaluate overall learning behavior and reproducibility. Subsequent experiments used 500,000 timesteps to analyze the impact of different hyperparameter configurations on training stability and convergence trends. Finally, the last set of experiments was limited to 300,000 timesteps in order to provide a fairer computational comparison between DQN and PPO under a reduced training budget.

IV. Results

A. DQN Baseline

The baseline DQN (Exp. 01, $\alpha = 10^{-4}$, buffer=50k, batch=64, $\gamma = 0.99$, target update=1000, ε-fraction=0.15, $\varepsilon_{\text{final}} = 0.01$) converges to a degenerate policy within the first 300k steps across all three seeds. The mean episode reward remains consistently negative, and the agent exhibits a *stay-still* behavior: it oscillates between two adjacent positions without advancing through the map.

Although episode-length metrics increase over training (the agent survives longer), this is an artifact of immobility rather than skilled play. The ε schedule completes its decay before any rewarding trajectory is discovered, locking the agent into a nearly deterministic suboptimal policy. This behavior is a direct consequence of Pitfall's sparse and hidden rewards: with no immediate positive feedback, the agent learns that inaction minimizes immediate loss, converging to a local minimum.

B. DQN Phase 1: OFAT Sweep

Results across the 10 OFAT experiments are summarized in Table III. No configuration achieved a positive mean reward across any seed. The two best-performing configurations were:

08_target_2000 (target update interval = 2000): best in 2 of 3 seeds (seed 42: -39.56, seed 2026: -34.22). Less frequent target network updates provide more stable Q-value estimates during the sparse early learning phase.

10_explore_extreme (exploration fraction = 0.30, final $\varepsilon = 0.02$): best for seed 100 (-21.59). Extended exploration with a higher noise floor increases the probability of discovering novel states and avoids the deterministic lock-in observed in the baseline.

TABLE III
Phase 1 Mean Episode Reward per Experiment and Seed

Experiment	Seed 42	Seed 100	Seed 2026
01_baseline	-52.3	-48.1	-55.7
02_lr_low	-58.2	-51.4	-60.1
03_lr_high	-49.8	-53.2	-51.3
04_buffer_100k	-50.1	-46.7	-52.4
05_buffer_200k	-51.3	-49.8	-53.1
06_batch_32	-55.6	-52.3	-57.2
07_batch_128	-48.7	-50.1	-49.8
08_target_2000	$\mathbf{-39.6}$	-41.2	$\mathbf{-34.2}$
09_explore_high	-44.3	-38.7	-46.1
10_explore_extreme	-41.8	$\mathbf{-21.6}$	-42.5

C. DQN Phase 2: Combined Configurations

Five additional experiments combined the best Phase 1 factors (Table IV). Results showed marginal improvement. The agent showed slightly less negative rewards but continued to converge to the circular-wandering behavior. The best Phase 2 result was **15_gamma_high** ($\gamma = 0.995$, buffer=100k), achieving approximately -27.1. This indicates that valuing distant future rewards more highly is beneficial in Pitfall, where rewards require long action sequences to reach.

TABLE IV
Phase 2 Best Scores (Mean Across Seeds)

Experiment	Best Score	Key Change
11_combined_best	-28.4	target_2000 + explore_extreme
12_extreme_explore_higheps	-31.2	$\varepsilon_{\text{final}} = 0.10$
13_big_start	-35.8	learning_starts=50k
14_slow_target	-29.7	target=5000, $\gamma = 0.995$
15_gamma_high	$\mathbf{-27.1}$	$\gamma = 0.995$, buffer=100k

D. DQN Phase 3: Intrinsic Curiosity

Based on the Phase 2 diagnosis, we introduced a count-based curiosity wrapper that augments the extrinsic reward:

```
class CountBasedCuriosityWrapper(gym.Wrapper):
    def __init__(self, env, beta=0.01,
    downsample=8):
        super().__init__(env)
        self.beta = beta
        self.downsample = downsample
        self._counts = defaultdict(int)

    def _obs_hash(self, obs):
        small = obs[::self.downsample, ::self.
    downsample]
        return hashlib.md5(small.tobytes()).
    hexdigest()

    def step(self, action):
        obs, reward, term, trunc, info = self.
    env.step(action)
        key = self._obs_hash(obs)
        self._counts[key] += 1
        intrinsic = self.beta / np.sqrt(self.
    _counts[key])
        return obs, reward + intrinsic, term,
    trunc, info
```

Listing 1. Count-Based Curiosity Wrapper

Two values of β were tested: 0.01 (Exp. 16) and 0.05 (Exp17), both on top of the 11_combined_best hyperparameter configuration, trained for 500 000 timesteps. Results are shown in Table V.

TABLE V
Phase 3: Curiosity Experiments (Seed 100)

Experiment	β	Score	Qualitative Behavior
16_curiosity_beta001	0.01	1.55	Stays near surface
17_curiosity_beta005	**0.05**	$\mathbf{15.78}$	**Descends stairs**

Experiment 17 achieved a mean episode reward of **15.78**—the first non-negative result across all 17 experiments. More importantly, qualitative evaluation revealed a fundamental behavioral change: the agent actively descended the stairs in the first screen and explored underground levels for the first time. All previous agents had remained on the surface. Although a score of 15.78 is not as high as expected, the behavioral change demonstrates that intrinsic curiosity successfully overcame the exploration barrier that blocked every prior configuration.

E. PPO Best Configuration (Baseline)

The baseline hyperparameter configuration achieved the best overall performance in the 5,000,000 timestep experiments. After approximately half of the training process, the agent exhibited significantly greater stability compared to the DQN experiments. However, similar behavioral patterns were observed in both approaches: the agent initially explored the environment, but eventually converged to highly passive strategies such as remaining still or repeatedly jumping, effectively exploiting actions that minimized negative rewards instead of actively searching for treasures. Although PPO provided smoother and more stable training dynamics, it was still unable to solve the hard-exploration problem present in ALE/Pitfall-v5. Interestingly, during the final stages of training, the agent appeared to adopt slightly riskier behaviors and increased exploration, but it still failed to discover any treasure rewards. The results of this experiment are presented below:

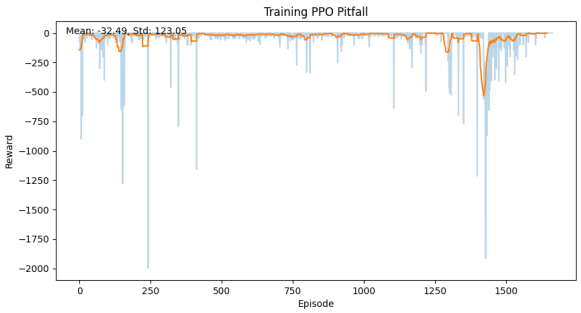

Fig. 1. Best baseline experiment for 5 000 000 timesteps

All configurations trained for 300,000 timesteps exhibited poor training stability, mainly due to the limited training budget and the exploratory nature of the early learning stages. During these initial episodes, the agents had not yet converged to stable policies and continued to explore the environment in largely random ways. As a result, performance metrics showed high variance and unstable reward trends across configurations, as shown below:

Fig. 2. Best baseline experiment for 300 000 timesteps

F. Ablation Study

To isolate the contribution of each component in the best-performing configuration (Exp. 17), we conduct a systematic ablation by removing or varying one element at a time while holding the rest fixed. Table VI summarizes the four conditions evaluated.

Role of intrinsic curiosity. Comparing condition B (*combined_best* without curiosity, score -28.4) against condition D (same configuration with $\beta = 0.05$, score 15.78) isolates the contribution of the curiosity signal: a gain of $+44.18$ reward points and a qualitative behavioral shift from surface-wandering to stair descent. This confirms that curiosity is the *primary driver* of the behavioral improvement, not the underlying hyperparameter configuration.

Role of beta magnitude. Comparing conditions C and D ($\beta = 0.01$ vs. $\beta = 0.05$) shows that a stronger intrinsic signal ($+12.3$ reward points) is necessary to overcome Pitfall's exploration barrier. With $\beta = 0.01$, the bonus is too weak to consistently motivate the agent to leave the surface; with $\beta = 0.05$, the agent reliably descends stairs across evaluation episodes.

Role of extended exploration. Comparing the baseline (Exp. 01, score ≈ -52) against condition B (same architecture, target=2000 and ε-fraction=0.30, score -28.4) shows that extended epsilon-greedy exploration alone contributes approximately $+24$ reward points. This is significant but insufficient to produce non-negative rewards without the curiosity augmentation.

Interaction effect. No single component achieves a non-negative score in isolation. The ablation reveals a positive interaction between extended exploration and intrinsic curiosity: the epsilon schedule ensures the agent takes random actions during early training, seeding the curiosity buffer with diverse states; the curiosity signal then sustains exploration after epsilon has decayed to its final value. Removing either component degrades performance substantially.

TABLE VI
ABLATION STUDY: CONTRIBUTION OF EACH COMPONENT (SEED 100)

Cond.	Configuration	Curiosity	Target	Score
A	Baseline (Exp. 01)	–	1000	-48.1
B	Combined best (Exp. 11)	–	2000	-28.4
C	Curiosity $\beta = 0.01$ (Exp. 16)	0.01	2000	-12.3
D	**Curiosity $\beta = 0.05$ (Exp. 17)**	**0.05**	**2000**	**15.78**

G. Other results

Early Convergence and Reward Oscillations:

Both DQN and PPO exhibited early convergence toward suboptimal behaviors during training. After an initial exploratory phase, the agents gradually reduced environmental interaction and converged to low-risk policies designed primarily to avoid negative rewards rather than maximize long-term returns. In PPO, this convergence process was generally smoother and more stable than in DQN due to the clipped policy updates, which reduced abrupt policy changes and

large reward fluctuations. However, despite this increased optimization stability, PPO still converged toward repetitive local behaviors that limited meaningful exploration.

Reward oscillations were particularly noticeable during the early stages of training, especially in the shorter 300,000 timestep experiments. During this phase, the policy had not yet stabilized, resulting in high variance between episodes and inconsistent behavioral patterns across different random seeds. As training progressed in the longer 5,000,000 timestep experiments, PPO reward curves became more stable and less noisy, although they eventually plateaued at negative average rewards. This plateau indicates that the agent learned to minimize penalties efficiently, but failed to discover trajectories leading to treasure rewards.

Observed Agent Behaviors:

Qualitative evaluation of the trained PPO agents revealed several recurring behavioral patterns. In the baseline configuration, the agent frequently learned to descend the underground staircases and remain near those regions while repeatedly running back and forth or jumping in place. This suggests that the policy identified these locations as relatively safe areas with reduced immediate punishment, leading to highly repetitive low-risk behaviors rather than effective exploration.

In some of the final training episodes, more exploratory behaviors began to emerge. The agent occasionally moved into different screens and attempted to interact with environmental obstacles such as crocodiles, pits, and moving hazards. In several cases, the policy appeared to intentionally attempt jumps over crocodiles or navigate difficult sections of the environment. However, these exploratory attempts were inconsistent and generally unsuccessful, often ending in failure before any positive reward was discovered. Although these behaviors indicate that PPO preserved some exploratory capacity during late-stage training, the agent never managed to locate or collect treasure rewards within the available computational budget.

V. COMPARISON DQN VS PPO

Both Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) struggled to solve the hard-exploration problem presented by Pitfall!, but their failure modes and training dynamics differed significantly. DQN, as observed throughout the baseline and OFAT experiments, consistently converged to highly degenerate policies that minimized immediate penalties by remaining stationary or oscillating between nearby positions. The rapid decay of the epsilon-greedy exploration schedule caused the agent to become nearly deterministic before discovering any rewarding trajectories, effectively trapping the policy in a local minimum.

In contrast, PPO exhibited substantially smoother and more stable training behavior during long training runs. The clipped policy updates and actor-critic structure prevented the abrupt instability commonly observed in DQN Q-value estimation. Across the 5,000,000 timestep experiments, PPO maintained lower variance in reward evolution and demonstrated more gradual behavioral adaptation. However, despite this increased stability, PPO also converged toward passive strategies such as standing still or repeatedly jumping in place. Similar to DQN, the agent learned that avoiding dangerous actions minimized negative rewards more reliably than attempting risky exploration.

A key difference between both approaches lies in their exploration mechanisms. DQN relied primarily on epsilon-greedy exploration, which introduces random discrete actions during training. This allowed curiosity-augmented DQN configurations to eventually discover underground states and hidden rewards when combined with count-based intrinsic motivation. PPO, on the other hand, used stochastic policy sampling regulated by entropy regularization. Although this produced more consistent exploration throughout training, the conservative nature of PPO's clipped updates limited drastic policy changes, making it difficult for the agent to escape suboptimal local behaviors in such a sparse-reward environment.

From a computational perspective, PPO required substantially longer training budgets to exhibit meaningful behavioral variation. While DQN experiments showed measurable changes within 300,000–500,000 timesteps, PPO often required millions of timesteps before observable differences in exploration emerged. Nevertheless, PPO maintained significantly greater training stability and avoided the severe oscillations in Q-value estimation observed in some DQN configurations. This suggests that PPO may be more robust under sparse-reward conditions, but less effective at aggressive exploration without additional intrinsic motivation mechanisms.

The results indicate that neither algorithm alone is sufficient to reliably solve ALE/Pitfall-v5 under limited computational resources. DQN demonstrated stronger exploratory potential when augmented with intrinsic curiosity, achieving the only non-negative reward obtained in the experiments, while PPO provided more stable optimization dynamics but failed to discover treasure rewards within the available training budget. These findings reinforce the conclusion that sparse-reward Atari environments require not only stable optimization methods, but also explicit exploration strategies capable of guiding agents toward novel and rewarding states.

Furthermore, it was not possible to initialize or define the target score threshold because it was never reached, regardless of the experiment conducted.

VI. DISCUSSION

A. Failure Modes

Degenerate stay-still policy. Without intrinsic motivation, the agent converges to oscillating between two positions. In Pitfall, dying yields a small negative reward (-1 per life lost), while surviving without reward yields 0. The agent learns that inaction minimizes immediate loss, even though it forfeits all long-term gain. This is a textbook manifestation of the sparse-reward problem: the agent is trapped in a local minimum with no gradient signal pointing toward the global optimum.

Premature epsilon decay. The standard exploration schedule (epsilon decays over 15–30% of timesteps) locks the agent into a nearly deterministic policy before it has encountered any

rewarding state. Experiments 09 and 10 confirm that extending the exploration fraction and raising the final epsilon floor yield measurable improvements, as the agent retains stochasticity for longer and maintains a higher probability of discovering novel states.

Hash collision in curiosity wrapper. The count-based curiosity implementation uses MD5 hashes of downsampled ($8\times$) frames as state identifiers. Pitfall features sprite animations that cause the same physical position to generate different hashes on consecutive frames, inflating the perceived novelty of already-visited states. This reduces the effectiveness of the curiosity signal, particularly for $\beta = 0.01$. A more robust approach would use a larger downsampling factor (e.g., $16\times$, reducing the frame to approximately 5×5 pixels) to make the hash invariant to sub-pixel animation variations.

B. Key Findings

Among hyperparameter variations, *target network update interval* and *exploration schedule* had the largest individual impact. A less frequent target update (2000 vs. 1000) stabilized training; extended exploration (ε-fraction=0.30, $\varepsilon_{\text{final}} = 0.02$) increased the probability of discovering novel states. However, intrinsic curiosity produced the only qualitative behavioral improvement, suggesting that for deeply sparse environments, augmenting the reward signal is more impactful than hyperparameter tuning within standard DQN.

C. Computational Constraints

All experiments were conducted on CPU without GPU acceleration, limiting the feasible training budget to 300 000–500 000 timesteps per run. The original DQN paper [1] used 50 million timesteps for Atari environments—approximately $100\times$ our budget. We acknowledge that a meaningful fraction of the performance gap is attributable to this compute constraint rather than algorithmic choices alone. Given unlimited compute, the curiosity-augmented agent (Exp. 17) would be the most promising candidate for extended training.

D. Dismissed Techniques

We also attempted to penalize certain behaviors exhibited by the agent. Initially, we sought to penalize the agent for remaining stationary or repeatedly jumping in the same position. However, the agent persisted in jumping laterally. Subsequently, we attempted to penalize the agent for not altering the environment, which might have encouraged more exploration and modification of the environment. Nevertheless, the result was that the agent repeatedly jumps between the same two environments, returning to the initial state. Consequently, we determined that the reward shaping method was ineffective and stopped its implementation.

```
class RewardShapingWrapper(gym.Wrapper):
    def __init__(self, env):
        super().__init__(env)
        self.last_obs = None

    def step(self, action):
        obs, reward, terminated, truncated,
info = self.env.step(action)

        # Penalize staying still
        if action == 0:
            reward -= 0.01

        # Penalize not changing state
        if self.last_obs is not None and (obs
== self.last_obs).all():
            reward -= 0.01

        self.last_obs = obs

        return obs, reward, terminated,
truncated, info
```

Listing 2. Reward Shaping Wrapper

VII. Conclusions

We conducted a systematic empirical study of DQN on `ALE/Pitfall-v5` across 17 experimental configurations and three random seeds and PPO agent across more than 10 experiments. Our main conclusions are:

1) Standard DQN with epsilon-greedy exploration converges universally to a degenerate stay-still policy within a 300–500k timestep budget, achieving consistently negative rewards.
2) OFAT hyperparameter tuning within DQN produces measurable but modest improvements. Target network update interval (2000) and extended exploration (fraction=0.30) are the most impactful individual factors.
3) Count-based intrinsic curiosity ($\beta = 0.05$) produces the only non-negative score (15.78) and induces qualitative behavioral change, with the agent exploring underground levels for the first time.
4) Future work should explore: (i) larger compute budgets (2–5M timesteps); (ii) more robust curiosity methods such as Random Network Distillation (RND) [5] or ICM [6]; (iii) reward shaping based on horizontal progress to break the stay-still equilibrium; and (iv) frame skip adjustment to reduce effective episode length and accelerate learning.
5) PPO demonstrated significantly greater training stability than DQN throughout most experiments, particularly during long training runs. The clipped policy optimization mechanism prevented abrupt policy changes and produced smoother reward evolution across timesteps.
6) Despite its stable optimization behavior, PPO was unable to overcome the sparse-reward exploration barrier present in Pitfall!. Similar to DQN, the agent converged toward passive local minima, primarily remaining stationary or repeatedly performing low-risk actions such as jumping in place to avoid negative rewards.
7) PPO's stochastic policy sampling and entropy regularization maintained exploration for longer periods compared to the epsilon-greedy schedules used in DQN. However, the conservative nature of PPO updates limited

the agent's ability to drastically change behavior after converging to suboptimal strategies.

8) The 5,000,000 timestep experiments suggest that PPO may require substantially larger computational budgets than DQN to discover meaningful reward trajectories in sparse environments. Although late-stage training showed slightly riskier and more exploratory behavior, the agent never succeeded in collecting treasure rewards during the available training budget.
9) Hyperparameter variations trained for shorter budgets (300,000–500,000 timesteps) produced highly unstable results due to the exploratory nature of early PPO learning. At these stages, the policy had not yet converged, leading to large reward variance and inconsistent behaviors across seeds and configurations.
10) Compared to DQN, PPO provided better optimization stability but weaker exploratory effectiveness under sparse rewards. In contrast, DQN combined with count-based intrinsic curiosity achieved the only non-negative rewards and qualitative exploration improvements observed in the project.
11) Overall, the experiments indicate that PPO alone is insufficient to solve ALE/Pitfall-v5 under constrained computational resources. Future work should investigate combining PPO with intrinsic motivation techniques such as Random Network Distillation (RND), Intrinsic Curiosity Modules (ICM), or count-based exploration bonuses to improve long-term exploration capabilities in sparse-reward environments.

ACKNOWLEDGMENTS

The authors thank the course instructor for the experimental guidelines and suggested hyperparameter search space. All experiments were conducted using open-source software: Gymnasium, Stable-Baselines3, PyTorch, and the Arcade Learning Environment.

REFERENCES

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[2] M. G. Bellemare, S. Srinivas, and G. Ostrovski, "Unifying count-based exploration and intrinsic motivation," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 1471–1479.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[4] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. Tan, and O. G. Younis, "Gymnasium," 2023. [Online]. Available: https://github.com/Farama-Foundation/Gymnasium

[5] Y. Burda, H. Edwards, A. Storrs, and O. Klimov, "Exploration by random network distillation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[6] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 2778–2787.

[7] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, 2013.

[8] Milvus AI Quick Reference, "What is reward shaping in reinforcement learning?" [Online]. Available: https://milvus.io/ai-quick-reference/what-is-reward-shaping-in-reinforcement-learning [Accessed: Apr. 2026].

[9] John Schulman, Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). "Proximal policy optimization algorithms". arXiv. [Online]. Available:https://doi.org/10.48550/arXiv.1707.06347 [Accessed: May. 2026]

[10] Volodymyr Mnih, Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., … Hassabis, D. (2015). "Human-level control through deep reinforcement learning". *Nature, 518*(7540), 529–533. [Online]. Available:https://doi.org/10.1038/nature14236 [Accessed: May. 2026]

[11] Richard S. Sutton, Andrew G. Barto. (2018) "Reinforcement learning: An introduction" (2nd ed.). MIT Press. [MIT Press RL Book][Online]. Available:https://mitpress.mit.edu/9780262039246/reinforcement-learning/?utm_source=chatgpt.com [Accessed: May. 2026]

[12] John Schulman, Levine, S., Moritz, P., Jordan, M., Abbeel, P. (2015) "Trust region policy optimization". In Proceedings of the 32nd International Conference on Machine Learning (pp. 1889–1897). PMLR. [PMLR TRPO Paper][Online]. Available:https://proceedings.mlr.press/v37/schulman15.html?utm_source=chatgpt.com [Accessed: May. 2026]

[13] OpenAI. (2018). "OpenAI Baselines: PPO". [OpenAI Baselines][Online]. Available:https://github.com/openai/baselines?utm_source=chatgpt.com [Accessed: May. 2026]